

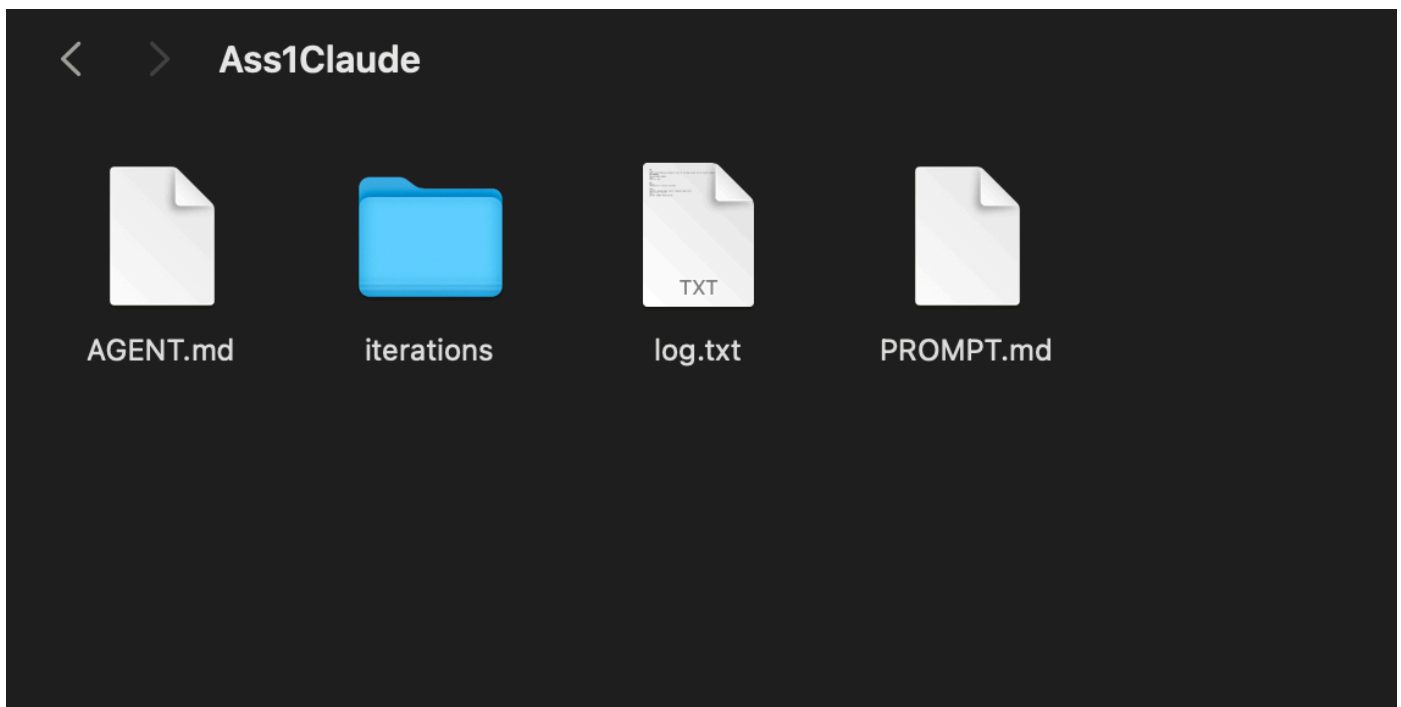
GenAI Code Comparison — Assignment 1

Frankie Tryban · EECS 348 Software Engineering I

How I set this up

I gave both models an identical starting point. Two work folders with the same contents:

- `PROMPT.md` — the assignment prompt, pasted in verbatim, unedited
- `AGENT.md` — completely empty
- `iterations/` — empty



```
//PROMPT.md
Objective
Write a C program that picks a secret number between 1 and 10.
The user gets up to 3 tries to guess the number.
Requirements
• Use a fixed secret number (for example, 7).
• Use printf to prompt the user and scanf to read their
guess.
• Use a while or for loop to let the user guess up to 3 times.
• Use if-else statements to check if the guess is too high,
too low, or correct.
• Stop the loop immediately if the user guesses the correct
number.
• Print a final win or lose message.
Example Output
text
Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 5
Too low! Try again.
Attempt 2/3. Enter your guess: 8
Too high! Try again.
Attempt 3/3. Enter your guess: 7
Correct! You win!
```

Then, in the Claude Code and Codex apps, I gave each one the same prompt, character for character:

```
Read AGENT.md and PROMPT.md. Write a C program that satisfies PROMPT.md.  
Save it to iterations/00-raw.c. Do not create or modify any other files.
```

I included the empty `AGENT.md` in the prompt on purpose. I plan to put standing instructions in that file later, and I wanted the wiring identical from the start, so that the only thing that ever changes between runs is the file's contents.

Claude Opus 5 (max) produced its output in **31 seconds**.

```
#include <stdio.h>

#define SECRET 7
#define MAX_TRIES 3

int main(void) {
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= MAX_TRIES; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, MAX_TRIES);
        scanf("%d", &guess);

        if (guess == SECRET) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < SECRET) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    if (!won) {
        printf("Out of tries! The number was %d. You lose.\n", SECRET);
    }

    return 0;
}
```

Codex GPT-5.6 (extra high) took 1 minute 3 seconds.

```
#include <stdio.h>

int main(void)
{
    const int secret_number = 7;
    int guess;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d/3. Enter your guess: ", attempt);

        if (scanf("%d", &guess) != 1) {
            printf("Invalid input. You lose!\n");
            return 0;
        }

        if (guess == secret_number) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess > secret_number) {
            printf("Too high! Try again.\n");
        } else {
            printf("Too low! Try again.\n");
        }
    }

    if (!won) {
        printf("You lose! The secret number was %d.\n", secret_number);
    }

    return 0;
}
```

First look

By eye I could see that Claude declared all its variables at the top of `main` and Codex declared its loop counter inside the `for` statement. I could see Codex used `const` and a longer variable name. I could see that neither one had a prolog header and neither had a single comment. I assumed they had the same time and space complexity, because they are both short programs that loop at most three times.

That was it. I had opinions about which one *looked* nicer with little facts about which one *was* better. Everything below is me going and getting facts.

How I got the facts

Working with Claude in a separate chat, I built a test setup that will test the code for me to get comparable facts.

Piped input. Instead of typing guesses by hand every time, I fed the programs their input:

```
printf '5\n8\n7\n' | ./claude
```

The `|` (pipe) takes the text on the left and hands it to the program on the right as if a person had typed it. This makes every test instant and repeatable, and it means I can test the same input on two different machines and compare honestly. One side effect worth knowing: a real terminal echoes the characters you type, and a pipe doesn't, so my transcripts show the program's output without the test user inputs typed out. The program's own output is unchanged. User typed input:

```
Guess a number between 1 and 10.  
Attempt 1/3. Enter your guess: 5  
Too low! Try again.  
Attempt 2/3. Enter your guess: 8  
Too high! Try again.  
Attempt 3/3. Enter your guess: 7  
Correct! You win!
```

Piped input:

```
Guess a number between 1 and 10.  
Attempt 1/3. Enter your guess: Too low! Try again.  
Attempt 2/3. Enter your guess: Too high! Try again.  
Attempt 3/3. Enter your guess: Correct! You win!
```

A test script.

Working with claude, I generated a short shell script (in github repo) that compiles both files with the command from the C Basics lecture (`gcc file.c -o file`), then runs each one through the same list of piped inputs — the assignment's example, a word instead of a number, no input at all, input that stops partway through, numbers outside 1–10, junk typed after a valid number, and a number too large to fit in an `int`. It prints the output and exit status of every run. It also compiles both files under `-Wall -Wextra`, under `-O2`, and under `-std=c89 -pedantic`, to see whether the compiler has anything to say that the plain command doesn't.

To test each code in the **actual grading environment**. I SSHed into `cycle1.eecs.ku.edu` and ran the entire script there:

```
x86_64 · Ubuntu 22.04 · gcc 11.4.0 · glibc 2.35
```

I also ran it on a Linux VM on my own laptop, which is aarch64 with the same gcc version. Running the same code on two different processor architectures ended up producing the single most important result in this report, which I would have missed entirely if I had only tested in one place.

Correctness

Both programs reproduce the assignment's example transcript **byte for byte**.

```
Guess a number between 1 and 10.  
Attempt 1/3. Enter your guess: 5  
Too low! Try again.  
Attempt 2/3. Enter your guess: 8  
Too high! Try again.  
Attempt 3/3. Enter your guess: 7  
Correct! You win!
```

On the inputs the example doesn't cover, they behave very differently. I checked with the script output rather than by eye.

Behavior on inputs the example never shows (cycle server)

5, 8, 7		
1, 2, 3	correct loss	correct loss
abc		
no input at all	burns all 3 attempts on garbage	ends game immediately
0 11 -5	accepted as guesses	accepted as guesses
5xy	xy	xy
4294967303	"Correct! You win!"	"Correct! You win!"

4294967303

Typing 4294967303 makes both programs congratulate you on the first attempt:

```
Guess a number between 1 and 10.
Attempt 1/3. Enter your guess: 4294967303
Correct! You win!
```

That number is $2^{32} + 7$. As my freind Lucas pointed out, an `int` holds roughly ± 2.1 billion, so this doesn't fit. It wraps around, like an odometer rolling from 999999 to 000000, and lands on exactly 7.

The part I found genuinely surprising is *where* it wraps. I wrote a tiny program to look:

```
scanf %d -> rc=1 value=7
```

`scanf` returns 1, which means "I successfully read one value," and it hands back 7. The overflow happens inside `scanf`, before my code ever sees the number. So this is not something you can catch by checking the value afterward — by the time you look at it, it really is 7. There is no evidence left that the user typed something else. Reproduced identically on both machines, so this is in the code, not in one computer.

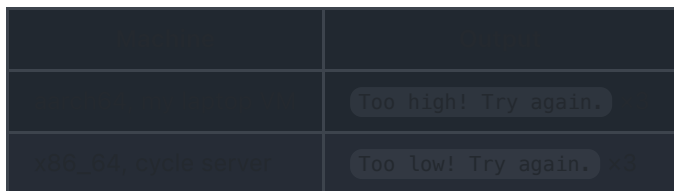
Claude only: the same program gave different answers on two computers

Claude calls `scanf` and throws away the return value:

```
scanf("%d", &guess);
```

When the input isn't a number, `scanf` writes nothing, and `guess` keeps whatever it already contained. On the first pass through the loop it never contained anything — `int guess;` reserves a slot of memory without setting it, so it holds leftover bytes from whatever used that memory before.

Same source file, same input `abc`:



I printed the value the program was actually comparing against 7, changing nothing but the size of the environment:

```
env pad=0b      -> guess = -663556032
env pad=64b     -> guess = -676323264
env pad=512b    -> guess = 1979629632
env pad=4096b   -> guess = 1487097920
```

The number this program tests against 7 is arbitrary. It changes with the machine and even with the environment on one machine. If it ever lands on 7, typing `abc` wins the game.

This is called **undefined behavior**. The C standard lists things it refuses to define, and reading a variable you never assigned is one of them — not "this is wrong," but "anything is allowed to happen here." That is why this matters more than any style complaint: a program whose output isn't determined by its source code cannot be tested, because passing once tells you nothing about the next run.

The assignment instructions warn that "the same code may produce different outputs in different environments." This is that sentence happening, and I only have proof because I compiled in both places.

There is a second thing going on in the same failure. What you type sits in a waiting line called the **input buffer**, and `scanf` reads from the front of it. When it can't parse `abc`, it doesn't remove it — it leaves it exactly where it was. So the next attempt reads the same `abc` and fails the same way, and so does the third. That is why one typo burns all three guesses.

Codex only: it checks, but one typo ends the game

Codex does check the return value:

```
if (scanf("%d", &guess) != 1) {
    printf("Invalid input. You lose!\n");
    return 0;
}
```

Checking is the right instinct, and it's the single biggest difference between the two files. Because Codex decides based on a value `scanf` actually returned, unlike Claude's, its behavior is **deterministic** — same input, same output, every machine, every time.

But the response is harsh. One mistyped character ends the game with attempts remaining. Typing `5xy` loses on attempt 2, because `xy` is still sitting in the input buffer from the first read. Codex has the same buffer problem as Claude; it just hides it by quitting before it can loop into it.

It also returns exit status `0` on that path. Zero conventionally means "this program succeeded," so it announces failure to the person and success to the operating system.

The compile command we were given hides the worst bug

gcc knows about Claude's unchecked `scanf`. It stays quiet anyway:

<code>gcc claude.c -o claude</code>	
<code>gcc -Wall -Wextra claude.c -o claude</code>	no
<code>gcc -O2 claude.c -o claude</code>	

```
claude.c:14:9: warning: ignoring return value of 'scanf' declared with
attribute 'warn_unused_result' [-Wunused-result]
```

Same result on both machines. The most dangerous defect in either program is invisible under the exact command we're told to use, stays invisible with warnings turned all the way up, and only appears once optimization is enabled. It's a warning, not an error, so even then it compiles and runs.

Execution Time

Both programs are $O(n)$ in time, where n is the number of guesses, and n is capped at 3. Neither loops over any data, so in practice the cost is constant.

100 runs of the losing path, piped input, on the cycle server:

Codex	0.138s	0.132s	0.063s

The user time is identical for both programs, to the millisecond. That's the number that measures the code itself, and it says these two programs cost the same. The entire 8 ms gap is in system time, on a server that other students are logged into at the same time. That measures how busy the machine was, not how good the program is.

So there is no meaningful execution-time difference between these two programs, and I'm not going to claim one. A static comparison agrees: `main` compiles to 74 instructions for Claude and 81 for Codex. Those extra 7 are the `scanf` check.

Space Complexity

Both reserve 48 bytes of stack for main

$O(1)$ for both. Every variable is a single fixed-size number, declared once, and nothing grows as the program runs.

Measured with `/usr/bin/time -v` on the cycle server, five runs each:

2	1580	1680
4	1604	1672
mean	1642	1654

There is no memory difference between these programs. The 12 KB between the means is about a tenth of how much either one varies on its own from run to run, making it not very informative into one code being better than the other.

This is the part of the assignment I learned the most from. My *first* measurement was a single run of each, and it showed Claude using 92 KB more than Codex. I was ready to write that down as a finding. Running it five more times made the difference disappear. If I had written the report from one measurement, I would have confidently reported something that isn't true. Most of that 1.6 MB is the C library loading anyway, not my program.

Maintainability

Codex won't compile under the older C rulebook

```
codex.c:11:5: error: 'for' loop initial declarations are only allowed in C99
or C11 mode
```

Codex writes `for (int attempt = 1; ...)`, which is legal in C99 and later but a hard error in C89. Claude declares its counter at the top of `main`, the older way, and compiles clean under both.

`const int` costs nothing here

Codex writes `const int secret_number = 7`; Claude writes `int secret = 7`. At the compiler level `-O0` both compile to the identical instruction (`mov w0, 7`). So the two have no real difference in space, but Codex sneaks on the `const` tag for free.

Naming

`secret_number` reads better than `secret`. Small, but it's free, and I'd rather read Codex's version in six months.

Branch order

Claude tests `guess < secret` first; Codex tests `guess > secret_number` first. For any valid guess these are identical — it's a coin flip the two models happened to call differently, which I think is worth noting on its own, because it shows how much of the difference between two models is arbitrary rather than reasoned.

It does have one real consequence. Because Claude is comparing garbage, whether that garbage prints "Too low" or "Too high" depends on the branch order.

Neither one documented anything

Neither raw file has a prolog header. No program name, no description, no inputs or outputs listed, no author, no date, and no statement of where the code came from. Neither contains a single comment.

Grading this against the rubric that's: the prolog comments, the in-line comments, and the collaborator disclosure — gone before the program is even evaluated. A lot of lost points. Neither model volunteered any of it, and neither was asked to. That's a real finding about working with these tools: they answer the question you asked and nothing else.

Verdict

I'm building on Codex's version, and one reason outweighs everything else: it checks the return value of `scanf`.

That single check is the difference between a program whose behavior is defined by its source code and one whose behavior isn't, and I have output from two different processor architectures proving Claude's isn't. Everything else I found — the harsh instant loss, the wrong exit status, the missing range check, the missing documentation — is a bug I can see, reason about, and fix. Undefined behavior isn't, because it doesn't fail consistently enough to trust a test that passes.

On time and memory the two are indistinguishable.

On maintainability Codex reads slightly better and Claude is slightly more portable (running on c89), and neither of those is close to decisive.

The overall thing I'm taking away: I couldn't tell any of this by reading the two files. The differences that turned out to matter went over my head until I compiled them, ran them against inputs the assignment never mentioned, and ran them somewhere other than my own laptop.

New and improved

Now I took this write up and fed it to a new Claude chat to generate a new AGENT.md. I also added 1 line to PROMPT.md.

```
Use this (analysis-draft.md) to create AGENT.md. LESS IS MORE. all the other attempts at this have lead to huge AGENT.md files
```

```
# AGENT.md
```

Standing rules for this folder. PROMPT.md says what the program must do. This file says how it must be written. Both

```
## Who is reading this code
```

Someone with about one year of Python and no C. Write for that reader.

```
## Comments
```

- Every line of code gets a comment.
- The comment says why the line is there, not what it says. ``int guess;`` is not "declare guess" – it is "reserve a
- The first time a piece of C notation shows up, explain it: ``#include`, `int main(void)`, `&guess`, `%d`, `break``.
- Where there was a real choice, name the option you didn't take and why this one won.
- Put no name, initials, model tag, or date on individual comments.

```
## Header
```

Every `.c` file opens with one block comment holding: program name, one-line description, inputs, outputs, author, da

```
## Correctness
```

- Give every variable a value on the line you declare it. Never read a variable you have not written to.
- Check what every input call hands back. ``scanf`` returns how many values it actually read – compare that to how ma
- On bad input, do not quit early unless PROMPT.md says to. Clear the leftover characters out of the input buffer a
- Exit status 0 means the program did its job. Return non-zero if it gave up.
- Declare variables at the top of a block, not inside ``for (...)`,` so the file compiles under old and new C alike.
- It must compile silently under ``gcc -Wall -Wextra -O2 file.c -o file``.

```
## Scope
```

Write only the file PROMPT.md names. Do not create, rename, or edit anything else. Do not add features nobody asked

Codex running GPT 5.6 Sol Extra High took 3 min and 25 sec to create this:

```

/*
 * Program name: Number Guessing Game
 * Description: Gives the player three chances to guess a fixed number from 1 to 10.
 * Inputs: Whole-number guesses entered at the keyboard.
 * Outputs: Prompts, comparison hints, input errors, and a final win or lose message.
 * Author: OpenAI Codex
 * Date: 2026-09-08
 * Help received from: GPT-5 and the GCC compiler.
 */

#include <stdio.h> /* Ask C's preprocessor to include the declarations for printf, scanf, and getchar before compili

int main(void) /* Start the program in main; int is its exit-status type, and void says main needs no arguments. */
{ /* Group all of the work that belongs to the main function. */
    const int secret_number = 7; /* Keep the answer fixed as required; const prevents an accidental change during t
    const int maximum_attempts = 3; /* Name the limit so the loop and messages cannot silently disagree about it. */
    int attempt = 1; /* Begin counting at one because that is how the player sees attempt numbers. */
    int guess = 0; /* Give the guess a safe initial value because C variables do not initialize themselves as Python
    int scan_result = 0; /* Save scanf's return value so input is trusted only when one requested value was read. */
    int input_character = 0; /* Hold each leftover character as an int because getchar also needs the special EOF v
    int has_extra_text = 0; /* Remember whether anything besides harmless spacing followed the integer on its line.
    int won = 0; /* Use zero for "not yet" so the final message can distinguish a win from three wrong guesses. */

    printf("Guess a number between 1 and 10.\n"); /* Explain the valid range before asking the player for input. */

    while (attempt <= maximum_attempts) /* Repeat until three valid guesses are used; invalid entries are re-prompt
    { /* Keep one complete prompt, validation, and comparison cycle together. */
        printf("Attempt %d/%d. Enter your guess: ", attempt, maximum_attempts); /* Use %d as a placeholder where p
        scan_result = scanf("%d", &guess); /* Ask for one integer; &guess gives scanf the address where it may sto

        if (scan_result == EOF) /* Detect an ended input stream, which cannot be repaired by clearing and re-prompt
        { /* Separate an unavailable input stream from ordinary invalid text. */
            printf("Input ended before the game could finish.\n"); /* Tell the player why no win-or-lose result can
            return 1; /* Send a nonzero status to the operating system because the program had to give up. */
        } /* Finish handling the end-of-input case before inspecting any characters. */

        has_extra_text = 0; /* Reset line validation because each new entry deserves an independent check. */
        input_character = getchar(); /* Read the next character so unwanted input does not remain for the next atte

        while (input_character != '\n' && input_character != EOF) /* Clear through the line ending while also check
        { /* Apply the same trailing-character rule to every character left on this input line. */
            if (input_character != ' ' && input_character != '\t' && input_character != '\r') /* Permit common spac
            { /* Record that the line was not just one integer with optional spacing. */
                has_extra_text = 1; /* Make entries such as 5abc invalid instead of accepting only their numeric p
            } /* Finish classifying this leftover character. */

            input_character = getchar(); /* Advance toward the line ending so no bad characters contaminate the nex
        } /* Stop clearing once the whole line is consumed or the input stream ends. */

        if (scan_result != 1) /* Reject the guess when scanf did not return the one successful conversion requested
        { /* Handle nonnumeric input without spending one of the player's three valid attempts. */
            printf("Invalid input. Enter an integer between 1 and 10.\n"); /* State both the type and range needed
            continue; /* Restart the while loop now; continue was chosen over nested code to keep invalid input seg
        } /* Finish recovery from a failed integer conversion. */
        else if (has_extra_text != 0) /* Reject a numeric prefix followed by other text because the complete entry
        { /* Give mixed input the same re-prompt behavior as entirely nonnumeric input. */
            printf("Invalid input. Enter one integer between 1 and 10.\n"); /* Clarify that only one clean integer
            continue; /* Preserve the current attempt number because no valid guess was supplied. */
        } /* Finish handling unwanted text after an integer. */
        else if (guess < 1 || guess > 10) /* Enforce both ends of the range after scanf has safely written the inte
        { /* Keep out-of-range numbers from counting as guesses, as the required re-prompt implies. */
            printf("Invalid input. Enter an integer between 1 and 10.\n"); /* Remind the player of the accepted bo
            continue; /* Ask for the same attempt again rather than reducing the three valid chances. */
        } /* Finish validating the range before comparing with the secret. */

        if (guess == secret_number) /* Test equality first so a correct guess can end the loop immediately. */
        { /* Keep the winning actions together. */
            won = 1; /* Preserve the outcome for the final-message decision after the loop. */
            printf("Correct! You win!\n"); /* Supply the required final win message as soon as the answer matches.

```

```

        break; /* Exit the loop immediately; break was chosen instead of waiting for its condition to be checked. */
    } /* Finish the correct-guess branch. */
    else if (guess < secret_number) /* Compare smaller guesses separately so the player receives the useful direction. */
    { /* Handle the too-low result without changing the secret number. */
        if (attempt < maximum_attempts) /* Mention another try only when at least one valid attempt remains. */
        { /* Use the encouraging form before the final attempt. */
            printf("Too low! Try again.\n"); /* Tell the player to raise the next guess. */
        } /* Finish the message used when another guess is available. */
        else /* Choose a truthful final-attempt message when no retry remains. */
        { /* Avoid promising another turn after the third valid guess. */
            printf("Too low!\n"); /* Still provide the required comparison hint for the last guess. */
        } /* Finish the last-attempt version of the low hint. */
    } /* Finish all handling for a guess below the secret. */
    else /* The validated guess can only be too high after equality and too-low cases have failed. */
    { /* Handle the remaining comparison without repeating an unnecessary greater-than test. */
        if (attempt < maximum_attempts) /* Mention another try only when the loop will actually allow one. */
        { /* Use the retry message before all attempts have been spent. */
            printf("Too high! Try again.\n"); /* Tell the player to lower the next guess. */
        } /* Finish the high message used when another guess is available. */
        else /* Choose the final-attempt wording after the third valid guess. */
        { /* Keep the last hint accurate without suggesting a fourth attempt. */
            printf("Too high!\n"); /* Provide the required comparison result for the final guess. */
        } /* Finish the last-attempt version of the high hint. */
    } /* Finish all handling for a guess above the secret. */

    attempt = attempt + 1; /* Spend exactly one attempt now that a valid but incorrect guess was processed. */
} /* End the guessing loop after a win or after three valid guesses. */

if (won == 0) /* Print the losing result only when the correct branch never set the win flag. */
{ /* Keep the final loss output out of successful games. */
    printf("You lose! The secret number was %d.\n", secret_number); /* Give closure and use %d to insert the final secret number. */
} /* Finish the required final lose message. */

return 0; /* Return 0 to tell the operating system that the game completed normally, whether won or lost. */
} /* End the main function and therefore the program. */

```

Test Program Results:

abc		
no input	instant loss, exit 0	clean message, exit 1
0 11		
-5	accepted	rejected
5xy	5	
5		accepted, whitespace tolerated

Still broken — the overflow: Attempt 1/3. Enter your guess: 4294967303 Correct! You win! I don't know if this is even possible to fix since the computer actually only sees 7

Cost of the improvements: stack frame 48 → 64 bytes, text 2473 → 2999. About 20% more code for validation that actually works.